

Python Programming Language

Learn Python from Basics to Real-World Applications — a beginner-friendly journey into the world's most popular programming language.

BEGINNER TO INTERMEDIATE

HANDS-ON EXAMPLES

REAL-WORLD READY

Made with GAMMA

Introduction to Python



What is Python?

Python is a high-level, interpreted programming language known for its clean, readable syntax. It was created by **Guido van Rossum** and released in **1991**.

Why Python?

- Ranked #1 on the TIOBE Index for multiple years
- Powers Google, NASA, Instagram, and Netflix
- Used across web dev, data science, AI, and automation

Made with GAMMA

Key Features of Python

Easy to Learn

Readable syntax that resembles plain English — ideal for beginners.

Cross-Platform

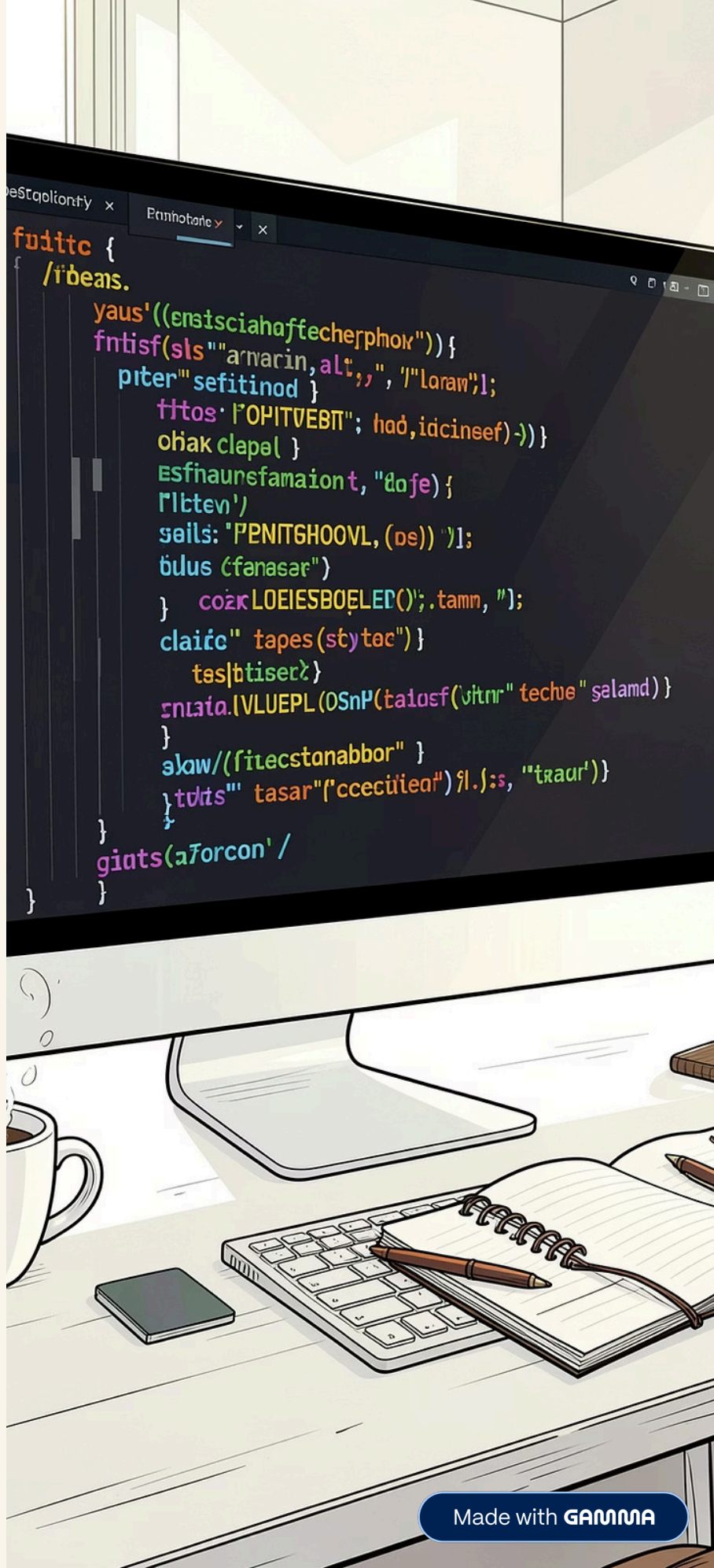
Runs on Windows, macOS, and Linux without modification.

Open Source

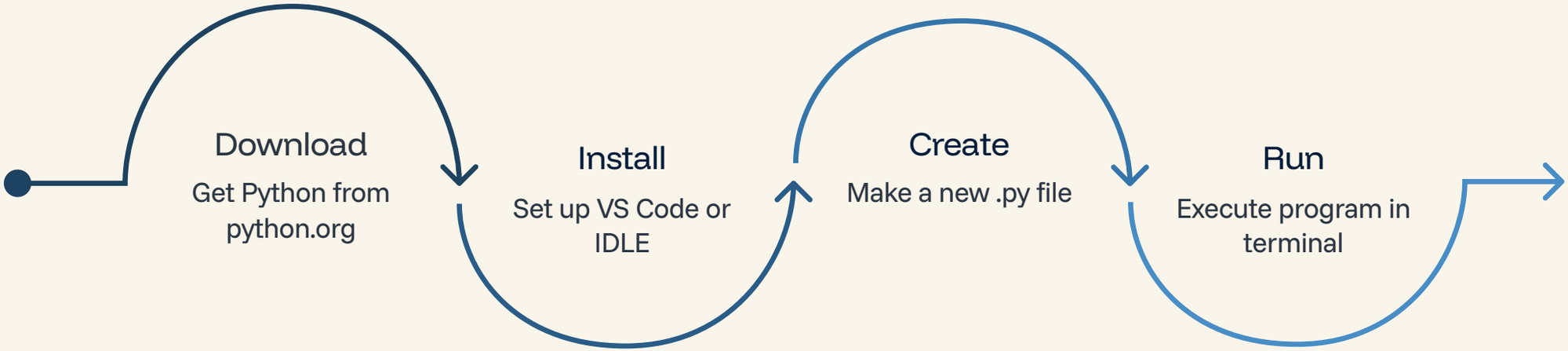
Free to use, modify, and distribute. Backed by a massive community.

Versatile

Supports OOP, scripting, web dev, data analysis, and AI.



Installation & Your First Program



Follow these four steps to set up your Python environment and run your very first program in minutes.

Verify Installation

```
python --version
# Output: Python 3.12.0
```

Hello, World!

```
print("Hello, World!")
# Output: Hello, World!
```

`print()` sends text to the screen — your first Python command!

Made with GAMMA

Python Syntax Basics



Core Concepts

Variables

`name = "Alice"` — no type declaration needed

Data Types

`int`, `float`, `str`, `bool`, `list`, `dict`

Indentation

Python uses **spaces**, not braces, to define code blocks

Comments

`# This is a comment` — ignored by Python

Made with GAMMA

Operators & Conditional Statements

Operators in Python

Arithmetic

`+ - * / % //`

Comparison

`== != > < >= <=`

Logical

`and or not`

Conditional Statements

```
age = 18
if age >= 18:
    print("Eligible to vote")
else:
    print("Not eligible")
```

Python evaluates conditions top-to-bottom. Indentation defines which lines belong to each block.

Made with GAMMA

Loops & Functions

Loops

```
# for loop
for i in range(1, 6):
    print(i)

# while loop
count = 1
while count <= 5:
    print(count)
    count += 1
```

Functions

```
def greet(name):
    return "Hello, " + name

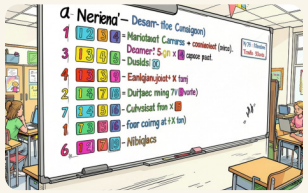
print(greet("John"))
# Output: Hello, John
```

Functions keep your code **organized and reusable**. Use `def` to define, `return` to send back a value.

> 💡 Always give functions descriptive names — it makes your code self-documenting!

Made with GAMMA

Data Structures: Lists, Tuples & Dictionaries



Lists

Ordered, **mutable** collections.

```
fruits = ["apple", "banana"]
fruits.append("orange")
```



Tuples

Ordered, **immutable** — can't change after creation.

```
coords = (10, 20)
print(coords[0])  # 10
```



Dictionaries

Key-value pairs for fast lookups.

```
person = {"name": "Riya", "age": 21}
print(person["name"])
```

Made with GAMMA

Popular Python Libraries

Python's ecosystem includes powerful libraries for every domain. Here are the most widely used:

Data & Analysis

NumPy · Pandas · Matplotlib · Seaborn

AI & Machine Learning

TensorFlow · Scikit-learn · OpenCV

Web & Automation

Requests · BeautifulSoup · Tkinter

Library Comparison

Library	Primary Use
NumPy	Numeric computing
Pandas	Data manipulation
Matplotlib	Data visualization
TensorFlow	Deep learning
Requests	HTTP web requests
Scikit-learn	Machine learning

Made with GAMMA

Live Examples: Pandas & Matplotlib

Pandas — DataFrames

```
import pandas as pd

data = {"Name": ["Ram", "Sita"],
        "Age": [20, 22]}
df = pd.DataFrame(data)
print(df)
```

Creates a table-like structure perfect for data analysis.

Matplotlib — Charts

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4]
y = [10, 20, 15, 30]
plt.plot(x, y)
plt.show()
```

Plots data points and renders an interactive chart window.

📊 Run these examples in Jupyter Notebook or VS Code with the `matplotlib` and `pandas` packages installed.

Made with GAMMA